Problem Set 3: Question 1

Monique and Harrison

April 4, 2022

Part A

The algorithm employed will be a modified version of a depth first search for a graph. Scenarios 1 to 3 each have a generally similar structure where scenario 2 and 3 only requires slight adaptation from scenario 1 to satisfy their respective requirements. Let n be the number of vertices or locations of a board configuration including start and end ports.

Scenario 1 asks that each location only be visited once, independent of it being a city or village. We begin by running a modified version of DFS where we start with a single node that is the start port. The first modification to the algorithm is, instead of never visiting a node again after it's been discovered, we will say that a node may be revisited as long as it isn't an ancestor of the current node. This means that each node may appear only once per route from leaf to starting node, however may appear multiple times in different routes from leaf to starting node. This means we no longer need to track visited nodes for the entire graph, rather, just the visited nodes to a given leaf on a given descent reducing the space complexity. The next modification is to count how many leaves of the produced tree are the end nodes. Finally, we don't actually need to build a DFS tree, so we do not make use of the timing heuristic again reducing space compelxity. We replace tracking the single parent with tracking all ancestral nodes required to get to the current node. But this is an array of length n and does not grow as we grow the DFS tree.

We chose to use a adjacency list for holding information on lists and edges. An adjacency list was favoured over a matrix since we rarely need to check for whether or not an edge exists. Rather, the algorithm must repetatively find the next edge for a given vertex, something a adjacency list is much better at.

All references to vertices were replaced with a numerical index such that we could have $\mathcal{O}(1)$ lookup times for the edges of a vertex. Ancestors on a given descent are tracked with a single, size n array of integers where each index of the array represents it's respective vertex and the integer represents how many times it has been visited on that descent. As we ascend, we remove an integer such that the recursive function is always updated with its correct ancestors. This, compared to alternative solutions such as a stack ADT, is much more conservative as it does not grow proportional to the depth of a descent.

Scenario 2 introduces a special property for cities. They are allowed to be visited an unlimited number of times. To achieve this, we take what was used for scenario 1, and modify it by taking in an additional array that asks as a marker for whether a given vertex is a city. If it is a city, we essentially ignore how many times it has been visited on that descent for that descent.

Scenario 3 requires a boolean flag to be passed with the recursive function that trackes whether or not a village has been visited twice. If so, then the boolean is set to true. On completing the part of the descent that included the village that was visited the second time, the boolean is reset back to false so that the next descent from the same node can be performed with having a ticket. In the case the boolean is true, we cannot visit a village that has already been visited, but if it's false, we can, and we simply set the boolean to true to prevent any more double visits during the descent.

Part B

To represent board configurations, we will employ a text file where a modified adjacency list is represented. Continuous segments of space will be treated as delimiters. Commas are purely aesthetic and solely for human readability. Colons are used to denote the vertex for which the rows following vertices form edges with. Each row in the text file will be interpreted as a list of vertices to form edges connecting to the vertex right before the colon. Each continuous segment of space will be interpreted as a break between vertices. The starting and ending node must be named "start" and "end" (case-sensitive) and cities will be represented by an asterisk beside the label of a vertex. Since a board configuration is undirected, the graph here will be too, and therefore, all starting vertices and destination vertices should have reciprocal relationships in the list.

Provided in the `example_board.txt` file is an example of the input format desribed above representing the sample board configuration given in the problem set handout.

Part C

See `Routes.py` for implementation of above described algorithm.

Part D

In addition to the sample board configuration, 3 other board configurations were provided to demonstate the correctness, and functionality of the algorithms.

`multi_city_board.txt` represents a board configuration with 2 cities. As a given precondition, no two cities can be adjacent to each other. Given the aforementioned assumption, we see that the algorithm does complete without infinitely looping and results in 2 routes for scenario 1, 2 routes for scenario 2, and 10 routes for scenario 3. Due to the simplicity of this board configuration, these results can be manually calculated to show they are correct.

`multi_start_board.txt` represents a board configuration where the starting port has multiple paths to both a village and a city. This test is meant to show that the start ports can support multiple paths without including paths that cross the start port as an intermediate location. This one results in 6, 13, and 37 for scenarios 1, 2, and 3, respectively. All scenarios are within manual tracing distance, however scenario 3 does require much more time.

`multi_end_board.txt` is meant to accomplish the same task as with the previous except for the end port. Again, to show that there is no issue with multiple paths entering the end port while not having the end port treated like a normal village. This test board results in 2, 3, and 5 for scenarios 1, 2 and 3 respectively. This board configuration was very simple and can be easily traced to show the previously mentioned results.

`no_city_board.txt` is a board where all nodes are adjacent to each other with the exception of the start node. This board is also special in that there are no cities. Therefore, it makes perfect sense that we would see 5, 5 and 13 for scenarios 1, 2 and 3 respectively, since scenario three is the only one a

village has the potential to be visited more than once. These results can be traced manually as the actual board configuration is relatively simple.

`pointless_cd_board.txt` is meant to better show that the end port is not a traversable node. We see that running the algorithms on this board configuration results in 2, 2 and 3 routes for scenarios 1, 2 and 3 respectively. If the end port was treated as a non special traversable point (village properties), we would see more routes for the third scenario as a ticket could hypothetically be used on the end node. This simple board can be traced manually to show the results are correct. Alternatively, the same board without c and d may be executed to show that the number of routes for all three scenarios are the same.